

Lesson:



Stack One Shot



Pre-Requisites

- Arrays
- Linked Lists

List of Concepts Involved

- Introduction to stacks
- Types of operations that can be performed on stacks
- Problems related to Stacks
- Overflow
- Underflow
- Array implementation of a stack
- Linked list implementation of a stack
- Advantages of array implementation of stack
- Disadvantages of array implementation of stack
- Advantages of linked list implementation of stack
- Disadvantages of linked list implementation of stack

Topic 1: Introduction to stacks

A stack is a linear data structure. The thing that separates it from other linear data structures is how it performs insertion and deletion operations. A pile of plates at a cafeteria would be a good example to understand how stacks work. If someone wants to take a plate from the pile, he/she takes the one at the top. Similarly, if a new plate has to be added to the pile, it is placed at the top of the pile. We call this type of operation LIFO (last in first out) or FILO (first in last out).

A stack is a linear data structure that follows, LIFO (or FILO) approach for insertion or deletion of objects.

Topic 2: Types of operations that can be performed on stacks

Main operations-

1. `push()`: inserts an element onto the top of the stack.
2. `pop()`: removes and returns the top element from the stack.

Auxiliary operations-

1. `top()`: returns the top element of the stack.
2. `size()`: returns the current size of the stack.
3. `isEmpty()`: returns true if the stack is empty, else returns false.
4. `isFull()`: returns true if the stack is full, else returns false.

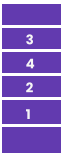
To understand them in a better way, let's assume we have a stack of maximum size 5 that currently contains 3 elements-



Let's try to call the auxiliary functions for the stack-

1. `top()`: it will return 4, the top element of the stack.
2. `size()`: it will return 3, the number of elements in the stack.
3. `isFull()`: it will return false, the number of elements in the stack(3) is not equal to the size of the stack(5) i.e. more elements can be added to the stack.
4. `isEmpty()`: it will return false, there are non-zero number of elements in the stack.

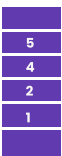
Now if we say `push(3)`. The value 3 will be added at the top of the stack.



Let's try to call the auxiliary operations again-

1. `top()`: this time it will return 3, the new top element of the stack.
2. `size()`: it will return 4, the size of the stack.
3. `isFull()`: it will return false, the stack is not full yet.
4. `isEmpty()`: it will return false, the stack isn't empty.

Similarly if we say `push(5)`. The value 5 will be added at the top of the stack.



Let's try to call the auxiliary operations again-

1. `top()`: this time it will return 5, the new top element of the stack.
2. `size()`: it will return 5, the size of the stack.
3. `isFull()`: it will return true this time, the number of elements in the stack is equal to the capacity of the stack.
4. `isEmpty()`: it will return false, the stack isn't empty.

Let's try the operation `pop()` this time. When we call the pop operation on the stack, the top element 5 is removed from the stack.



If we call it again, 4 will also be removed.



If we call it 3 more times, the stack will become empty



Let's try to call the auxiliary operations-

1. `top()`: since the stack is empty, this operation isn't valid. In such cases, we can throw an Exception stating that the stack is empty.
2. `size()`: it will return 0, the size of the stack.
3. `isFull()`: it will return false, there are no elements in the stack.
4. `isEmpty()`: it will return true, there are no elements in the stack.

Topic 3: Stacks in Java Collection Framework

Stack class can be found in the Java Collection framework that models and implements a Stack data structure. The class is based on the basic principle of last-in-first-out. In addition to the basic push and pop operations, the class provides three more functions of empty, search, and peek.

Declaration

Syntax for declaring a stack:

```
Stack<DataType> stack = new Stack<DataType>();
```

Example:

```
Stack<Integer> st1 = new Stack<Integer>();
```

This creates a stack that can store integer values, and the name of the stack is st1.

```
Stack<Character> st2 = new Stack<Character>();
```

This creates a stack that can store character values, and the name of the stack is st2.

List of member functions

Function	Use
<code>push(object)</code>	Inserts an element at the top of the stack
<code>pop()</code>	Removes and returns the top element of the stack. An 'EmptyStackException' exception is thrown if we call <code>pop()</code> when the invoking stack is empty.
<code>peek()</code>	Returns the top element of the stack without removing it from the stack
<code>size()</code>	Returns the number of elements present in the stack
<code>empty()</code>	Returns if the stack is empty or not
<code>search(object)</code>	It determines whether an object exists in the stack. If the element is found, it returns the position of the element from the top of the stack. Else, it returns -1.

Example

Output

Top element of the stack: 1
Is the stack empty? false
Size of the stack: 1

Top element of the stack: 2
Is the stack empty? false
Size of the stack: 2

Top element of the stack: 3
Is the stack empty? false
Size of the stack: 3

Stack before popping an the top element: [1, 2, 3]
Popped element: 3
Stack after popping an the top element: [1, 2]

First occurrence of 2 is at index 1

Topic 4: Problems related to Stacks

Q1. Copy contents of one stack to another in the same order.

Code

Explanation

1. Create a temporary stack.
2. Copy the elements of the original stack into the temporary stack. By default the order of the elements in the new stack will be reversed.
3. To create the new copy stack, copy the elements of the temporary stack to the new stack. Also push the same elements into the original stack to maintain its original state.

Q2. Insert an element at the bottom of the stack.

Code

Explanation

1. If the size of the stack is 0, simply add the value to the stack.
2. Remove the top element from the stack.
3. Call the recursive function.
4. While returning, push the elements back into the stack as they were.

Q3. Display reverse stack recursively.

Code

Explanation

1. If you've reached the last element, return.
2. Take out the top element of the stack. Print its value.
3. Call the recursive function function.
4. Insert back the top element to recreate the stack.

Q4. Remove the element from the bottom of the stack.

Code

Explanation

1. For base case, simply remove the element from the stack.
2. For every other call-
 - a. Remove the element from the stack.
 - b. Call the recursive function.
 - c. Insert the element back into the stack.

Q5. Reverse the stack.

Code

Explanation

1. Recursively remove the elements from the stack.
2. When returning back from the base case, push back each element into the stack. This time insert each element at the bottom of the stack.

Q5. Reverse the stack.

[Code](#)

Explanation

1. Recursively remove the elements from the stack.
2. When returning back from the base case, push back each element into the stack. This time insert each element at the bottom of the stack.

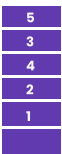
Topic 5: Overflow

It is the name of the condition when we try to insert elements into our stack more than it can hold. For example,

1. We create an integer stack of max size 5.



2. We insert 5 values into the stack 1→5.



3. Now even though we have already filled the entire stack, we try to fill in more elements into it.

Topic 6: Underflow

It is the name of the condition when we try to pop elements from an empty stack. |



In such a scenario, we can throw an Exception stating there is an underflow.

Topic 7: Array implementation of a stack

In the array implementation of a stack, we use an array to represent the stack. The elements are added into the stack in the increasing order of the indices of the array. The last element inserted in the array is considered the top of the stack and is present at the highest index. |



We start by declaring 3 things-

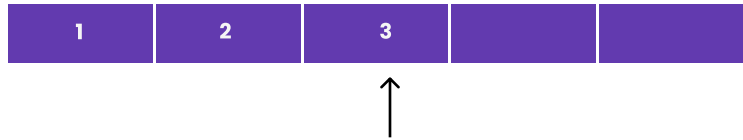
1. An integer variable to store the maximum size/capacity of the stack. Let's call it 'capacity'.
2. An array to represent the stack. Let's call it 'st'.
3. An integer variable denoting the position of the top element of the stack. Let's call it 'top'. We initialise it with -1, to denote that the stack is empty.

Now let's implement the stack functions-

top()

Since we already have a variable to represent the index of the top element of the stack. We can directly return the value at that index.

If the **top** is equal to -1 i.e. there is no element in the stack, we can give an error saying that the stack is empty. For instance, We have an array of max capacity 5, with 3 elements in it.



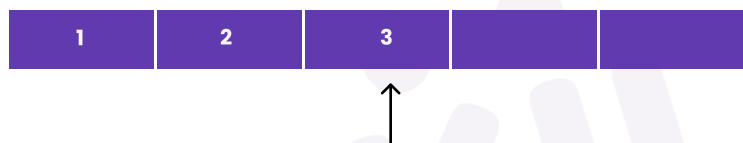
If we want to get the top value of the stack, we can directly print the value at the top index i.e. 3.

[Code](#)

Time Complexity: $O(1)$

size()

Since there is 0 based indexing in the array and the elements are inserted into the stack in the increasing order of the array indices, the size of the array will be the position of top variable + 1. For instance, We have an array of max capacity 5, with 3 elements in it.



The size of the stack = $2(\text{top}) + 1 = 3$.

[Code](#)

Time Complexity: $O(1)$

isEmpty()

If the value of **top** is -1, we can return true. Else it will be false.

[Code](#)

Time Complexity: $O(1)$

isFull()

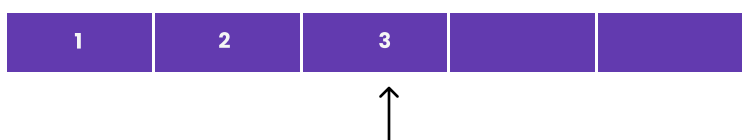
If the size of the stack is equal to the capacity of the stack, we can return true. Else it will be false.

[Code](#)

Time Complexity: $O(1)$

push()

1. The first thing we need to do before inserting an element into the stack is to check if the stack is full or not. We can do that with the **isFull()** function.
 - a. If the stack is already full, we cannot add a new element to the stack. So, we can give the overflow error and return.
 - b. Else we can increment the value of **top** and place our new value at that position. For example, We have an array of max capacity 5, with 3 elements in it.



$\text{top} = 2$ (it represents the index of the top element, 0-based indexing)

Let's insert an element into the stack.

First we increment the value of $\text{top} \Rightarrow \text{top} = 3$.



We place the new value at the index denoted by top .



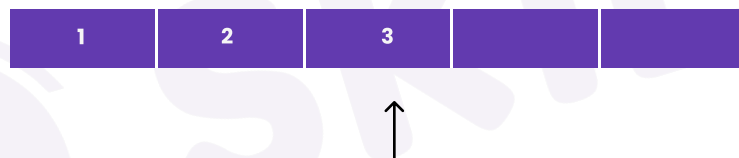
[Code](#)

Time Complexity: $O(1)$

pop()

1. To remove an element from the stack, we start by checking if the stack is empty or not. We can do that using the `isEmpty()` function.

- If the stack is already empty, we can't remove any element from the stack. So we can give an underflow error and return.
- Else we can just decrement the value of top . For example, We have an array of max size 5 that contains 3 elements in it.



$\text{top} = 3$

Let's pop an element from the stack. To do this we'll just decrement the value of top .



Now even if we have the value 3 present in our array as an element, by moving the top one place backward, we don't consider it a part of our stack. Our stack only consists of elements from index 0 to index top (both inclusive).

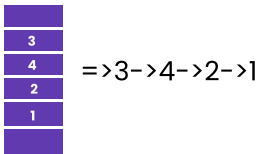
[Code](#)

Time Complexity: $O(1)$

[Stack class Code](#)

Topic 8: Linked list implementation of a stack

In the linked list implementation of a stack, we use a linked list to represent the stack. Each node of the linked list represents the element of the stack. The head of the linked list acts as the top of the stack and thus the insertion and deletion operations of the stack are performed at the head of the linked list.



Initially we create 2 things-

1. A node to represent the head of the linked list.
2. A variable to store the current size of the linked list.

Now let's implement the stack functions-

top()

Since the head of the linked list represents the top element of the stack, we can directly return the value stored in the head.

If the head is pointing to NULL i.e. there is no element in the stack, we can give an Exception stating that the stack is empty. For instance,

[Code](#)

Time Complexity: $O(1)$

size()

Since we have already created a variable to store the size of the linked list, we can return its value directly.

[Code](#)

Time Complexity: $O(1)$

isEmpty()

If the head of the linked list points to NULL, we can return true. Else it will be false.

[Code](#)

Time Complexity: $O(1)$

isFull()

In the case of the linked list representation, the amount of nodes that we can add to the linked list depends upon the memory limit of the system. So you'll automatically get an allocation failure message. In other words we don't need to create a separate function for it.

push()

The insertion of an element in the stack will be the same as the insertion of a node at the beginning of a linked list.

Steps-

1. Make a new node to insert in the list and assign it to a variable.
2. Make this new node point to the head of the linked list.
3. Make this new node the head of the list.
4. Increment the stack size.

[Code](#)

Time Complexity: $O(1)$

pop()

To pop an element from the stack, we just need to remove the head of the linked list.

Steps-

1. If the head of the list is pointing to NULL, give the underflow error and return.
2. Make 2 temporary variables to store the head and its value.
3. Make the head variable point to the second element of the list.

4. Return the value of the removed node.

[Code](#)

Time Complexity: $O(1)$

[Stack class Code](#)

Topic 9: Advantages of array implementation of stack

For each element only 1 block of space is needed.

Topic 10: Disadvantages of array implementation of stack

Due to the fixed size of the array, the overflow array can be encountered easily.

Topic 11: Advantages of linked list implementation of stack

Since each element is present at an independent memory location, running out of memory is nearly impossible.

Topic 12: Disadvantages of linked list implementation of stack

Storing each element requires additional memory for the pointer.